Federated Meta-Learning on Graph for Traffic Flow Prediction

Xinxin Feng, *Member, IEEE*, Haoran Sun, Shunjian Liu, Junxin Guo, and Haifeng Zheng, *Senior Member, IEEE*

***Abstract*—Traffic flow is considered as a critical feature of intelligent transportation systems (ITS). Accurately forecasting future vehicular volumes is an effective means of mitigating traffic congestion. However, the nonlinear and complex traffic flow characteristics make the traditional approaches unable to achieve satisfactory prediction performance. Although existing methods based on deep learning models have improved the accuracy of traffic flow prediction, the spatio-temporal features of traffic flow data are still not fully explored. Moreover, existing methods pay little attention to the task of training models in a decentralized environment where data are distributed across multiple clients. To solve the problems mentioned above, we propose a novel network model called Graph Transformer Attention Network (GTAN) for traffic flow prediction, which can effectively extract traffic flow's temporal and spatial characteristics by considering all node locations' information in the traffic networks. Then, we propose a training strategy called Graph Federated Meta-learning (FedGM), solving the problem of topological heterogeneity by combining meta-learning and federated learning, to achieve an optimal initialization model which can quickly adapt to different traffic networks under low communication cost. Finally, the experimental results on a real data set show that the GTAN model has better prediction performance and faster meta-training speed. The model trained by FedGM can quickly adapt to different graph-structured data and achieve high accuracy.**

***Index Terms*—Traffic flow prediction, graph transformer networks (GTANs), federated meta-learning, topological heterogeneity.**

I. Introduction

AS AN essential part of ITS, traffic prediction has significant meanings for urban traffic management. Predicting future traffic flow on roads can help individuals plan their travel routes, and predicting average road speeds can provide roughly estimated arrival times for travelers. Analyzing and processing past traffic data in a city can help urban traffic managers make scientific judgments about future traffic conditions and proactively alleviate traffic pressure in some areas. Therefore, accurate traffic prediction technology can provide cities with higher-level traffic management services, reducing the economic losses caused by accidents and promoting faster and more efficient urban development.

Traffic flow data exhibits spatial correlation, where sensors on the same road tend to capture similar traffic data distributions. Traffic flow data also shows temporal correlation, where traffic volume at adjacent time intervals may exhibit close relationships. Capturing the complex spatio-temporal correlations in traffic flow data can improve predictive accuracy. In recent years, some methods have combined graph neural networks (GNNs) with RNN-based techniques to better leverage spatial information in traffic data and have achieved significant performance improvements. For example, a Diffusion Convolutional Recurrent Neural Network (DCRNN) was proposed in [1] to capture the spatial and temporal correlations of traffic data and a Spatial-Temporal Graph Convolutional Network (ST-GCN) was presented in [2] that combines graph convolution and time convolution to capture the spatial-temporal correlation of traffic flow. However, graph convolutional neural networks rely on fixed adjacency matrices, making it challenging to extract complex spatio-temporal correlations. Additionally, training deep learning models relies heavily on data, while traffic data would be held by different organizations, such as government agencies and corporate enterprises. Therefore, sharing data among organizations is difficult due to privacy and commercial interests reasons, resulting in the impossibility for organizations of training effective models collaboratively.

With the advent of federated learning, the above problems have been greatly solved. By regarding organizations as clients, federated learning allows the clients to collaborate to build an effective model while maintaining local data to protect data privacy. However, each organization's local data varies in time, space and sources, leading to the problem of data heterogeneity. The data heterogeneity problem here mainly includes two aspects, one is the heterogeneity of the graph topology caused by the different deployment locations and numbers of sensor nodes in each region, and the other is non-independent and identical data distribution caused by the different size of the traffic flow data in each region. Data heterogeneity mainly affects federated learning in two aspects: firstly, it impacts the performance of federated learning, lowering the convergence speed and decreasing the accuracy of the final model. Secondly, since the data distribution and spatio-correlation in each client is dissimilar, a single global model can only meet some of the

Manuscript received 18 January 2024; revised 11 May 2024; accepted 30 July 2024. Date of publication 12 August 2024; date of current version 19 December 2024. This work was supported in part by the Natural Science Foundation of China under Grant 61971139, in part by the Major Science and Technology Program of Fujian Province under Grant 2022HZ026007, and in part by the Foundation of Fujian Province under Grant 2021J01576. The review of this article was coordinated by Prof. Nan Cheng. *(Corresponding author: Haifeng Zheng.)*

The authors are with the Fujian Key Lab for Intelligent Processing and Wireless Transmission of Media Information, College of Physics and Information Engineering, Fuzhou University, Fuzhou 350108, China (e-mail: fxx1116@fzu.edu.cn; 221127183@fzu.edu.cn; 470593980@qq.com; 371129248@qq.com; zhenghf@fzu.edu.cn).

Digital Object Identifier 10.1109/TVT.2024.3441759

client's needs, weakening the motivation for participating clients in federated learning. Many recent studies have attempted to address data heterogeneity in federated learning [3]. Though these approaches can overcome the problem to some extent, handling the data heterogeneity of traffic flow data represented as graph-structured datasets takes time and effort.

To address the above issues, we propose a novel traffic flow prediction model to fully exploit the sptio-temporal correlation of traffic data and introduce a graph-based federated meta-learning method to conduct optimal personalized models for different organizations. The main contributions of our work are as follows.

- We propose a novel model called Graph Transformer Attention Network (GTAN) by combining Transformer with GCN for traffic prediction. Compared with the other state-of-the-art methods, the proposed model is able to capture the spatial and temporal features of traffic flow more efficiently and achieve faster convergence.
- We propose graph federated meta-learning strategy FedGM, which implements meta-learning algorithms based on federated learning to learn an optimal initialization model that can quickly adapt to different graph topologies.
- We evaluate the proposed algorithm on the real-world traffic flow dataset. The results show that GTAN has better prediction accuracy and lower computational complexity, while FedGM can accelerate the convergence speed of meta-training and improve the adaptability of the trained meta-model to different graph topologies.

The remainder of this paper is organized as follows. Section II covers the literature on traffic prediction and federated learning. Section III describes the problem this paper aims to address, and then we detail the proposed method in Section IV. In Section V, we evaluate the predictive performance of the proposed method. Finally, we conclude the paper in Section VI.

II. Related Work

A. Traffic Flow Prediction

Traffic flow prediction is an important component of intelligent transportation systems (ITS) and has been a focus of researchers for many years. In the early days, researchers used methods based on traditional time series analysis models, such as History Average Model (HA) [4], Autoregressive Integrated Moving Average Model (ARIMA) [5], Vector Auto Regression Model (VAR) [6], and their related extended models, which were widely used time series methods. However, they cannot capture the nonlinearity and correlation between large-scale traffic data or complex spatio-temporal patterns, resulting in poor traffic flow prediction performance.

With the increasing attention given to machine learning concepts, methods such as K-nearest neighbour (KNN) and Support Vector Regression (SVR) have also been employed in traffic flow prediction. Although machine learning methods have demonstrated their advantages in processing complex data and achieving better prediction results compared to traditional methods, the complex spatial and temporal characteristics of traffic flow data still significantly impact the performance of machine learning methods. In addition, excellent machine learning methods often require large-scale feature engineering to pre-process traffic data, resulting in higher application costs.

Currently, deep neural networks have received attention from researchers due to their ability to extract deep features and learn such as DNN-BTF [7], which makes full use of weekly/daily periodicity and spatial-temporal characteristics of traffic flow. In recent years, traffic flow prediction methods have rapidly developed based on deep learning. For the complex spatio-temporal correlation in traffic flow sequences, some researchers have used recurrent neural networks (RNN) such as DMVST-Net [8] and LGNet [9] to capture the temporal characteristics of traffic flow data and make future predictions. With the improvement of LSTM, many models based on LSTM have been applied to traffic flow prediction tasks, in EnLSTM-WPEO [10], five different kinds of LSTM models with different time lags are separately learn the relationship of traffic flow. STA-LSTM [11] combines LSTM with spatiotemporal attention mechanisms to achieve interpretability in predicting vehicle trajectories. In addition, there are a number of models that also apply LSTM to the field, such as MATURE [12], AARGNN [13], GWO-SMOTE [14], and LSNet [15]. Furthermore, some methods such as CAS-CNN [16], TCN [17], TrafficGAN [18], and DeepGLO [19] use convolutional neural networks (CNN) to process longer sequences in less time and make predictions. Not only that, there are also models that combine LSTM and CNN to accomplish the traffic flow prediction task, e.g., in L-CNN [20], the CNN subnetwork is used to extract spatial dimension features and LSTM is used for temporal dimension and embedding.

However, while these methods are powerful in handling time series prediction problems, they cannot effectively utilize the spatial relationships among nodes in the traffic network. Therefore, some methods that combine graph neural networks (GNN), which are excellent at processing spatial features with methods that handle temporal features, have significantly improved performance in recent years. For example, Yu et al. proposed the Spatial-Temporal Graph Convolution Network (STGCN) [2], which captured the spatio-temporal correlation of traffic flow sequences by combining GCN and CNN. Based on STGCN, Guo et al. proposed the Adaptive Spatial Temporal Graph Convolution Network (ASTGCN), which added a spatial attention mechanism to capture the dynamic spatio-temporal correlation of traffic flow sequences more accurately [21]. To solve the problem of GCN being sensitive to time series, Song et al. proposed the STSGCN, which pre-stacked multiple adjacency matrices and used graph convolution operation to aggregate node information from multiple time segments to extract complete spatial-temporal correlation information and improve model prediction accuracy [22].

As a recently popular network structure, the transformer has attracted many researchers to apply this emerging technology to traffic flow prediction problems. STGNN achieved good prediction performance by combining transformer and GCN-GRU networks [23]. Xu et al. proposed a spatial transformer model (STTN), which used two transformers and GCN-GRU modules to process the temporal and spatial correlations of traffic flow

sequences for traffic flow prediction and achieved excellent prediction performance [24]. Furthermore, FDSA-STG [25] introduced spatial graph attention component, temporal graph attention component, and fusion layer to adapt to the structural relationship of transportation networks to dynamically integrate the correlation in spatial, temporal, and periodic features of traffic data.

Although deep learning-based methods have made great progress compared to previous methods, separately extracting time and spatial correlations ignores the complex dynamic characteristics of spatio-temporal relationships, and some models have slow computation speed and high application costs. Therefore, exploring an efficient method that can simultaneously extract spatio-temporal correlations in traffic flow sequences is necessary.

B. Federated Learning

Optimized algorithm for federated learning efficiency: Zhao et al. proposed having each client upload a subset of non-private data, collectively forming a public dataset. Before training, clients download the public data mix and incorporate it into their local dataset to alleviate the problem of imbalanced local data distribution. Their work suggests that clients only need to share 10% of their data to achieve approximately a 30% increase in accuracy [26]. Fraboni et al. suggested using clustering methods to group clients and selecting clients from different groups to participate in training during each round. They proposed clustering methods based on sample volume and similarity and demonstrated that sampling can lead to smoother convergence. Their method does not require additional client burden and can be combined with existing privacy protection and model compression technologies [27]. Tian et al. proposed Fedprox, which adds a second-order norm constraint term into local training to alleviate the bias of gradient direction in local training. They incorporated the second-order norm between the local model update and the global model as a loss term into the local loss to optimize, thereby limiting the deviation between the local and global models [28]. Chen et al. proposed FedBE [29], which aggregates the locally trained models into an ensemble learning model and compresses the ensemble model into a training model using knowledge distillation with unlabeled data held by the server. This work treats the models uploaded by clients as weak classifiers to aggregate a robust classifier, which can effectively utilize the differences between local models.

Personalized techniques in federated learning: Due to the often heterogeneous data owned by clients participating in federated learning, a single global model can only satisfy the needs of some clients. Personalization techniques in federated learning aim to train personalized models suitable for each client. Smith et al. proposed federated multi-task learning, treating the model training of each client as a separate task, and developed MOCHA to address the heterogeneity of data and communication efficiency in federated learning. Federated multi-task learning captures the inter-task relationships to benefit each task while retaining their distinct characteristics, leading to high-quality personalized models [30]. Fallah et al. proposed federated meta-learning, treating the training of global models in federated learning as meta-training and the subsequent personalization process as meta-testing. In federated meta-learning, the goal of federated learning is not to train a prediction model but to train a meta-learning model that guides the client training process. Heterogeneous data enables the meta-learning model to learn generalizable meta-knowledge to quickly adapt to different data [31], [32]. Shamsian et al. proposed joint training of a hyper-network, with the input being a unique embedding vector for each client and the output being the model weights generated for each client. During training, each client receives its network parameters for prediction and gradient computation, and the hyper-network is retained on the server, where it receives gradient-optimized parameters uploaded by clients [33].

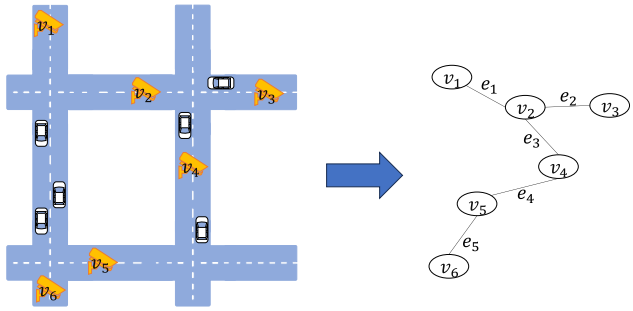

Fig. 1. A topology map of the traffic network.

Unlike the existing work, we apply federated learning to traffic prediction tasks, focusing on the heterogeneity of graph topology faced by graph structure data, and propose FedGM to adapt to different graph topologies quickly.

III. Problem Statement

A. Traffic Flow Prediction on Transportation Networks

Since traffic flow data exhibit spatial correlation, it can be constructed as graph data. As shown in Fig. 1, in a traffic area with several sensors, the topology graph of the traffic network is represented by $V = (v_1, \ldots, v_6)$ and $E = (e_1, \ldots, e_5)$, where V represents the sensor nodes in the area, and E means the connectivity between sensor nodes. The actual distance between two sensors determines the value of e_i; the larger the distance between two nodes, the smaller the value of e_i. Given a traffic network $G = (V, E)$ and historical traffic flow $X = (x_1, x_{2,\ldots,}x_T)$, the goal of traffic flow prediction is to create a model that takes T time steps of traffic flow data as input and predicts the future traffic flow values $X^\tau = (x_{1+\tau}, x_{2+\tau,\ldots,}x_{T+\tau})$ for the next τ time steps. Here, x_t represents the traffic sensor node data set at time t, represented as a vector.

B. Federated Learning on Transportation Networks

Consider the following scenario: different organizations deploy sensors on roads in different regions to collect traffic flow data. As shown in Fig. 2, based on the collected data, each region can accomplish tasks such as route optimization and navigation, as well as traffic congestion monitoring and management. However, more than the amount of data held by each organization

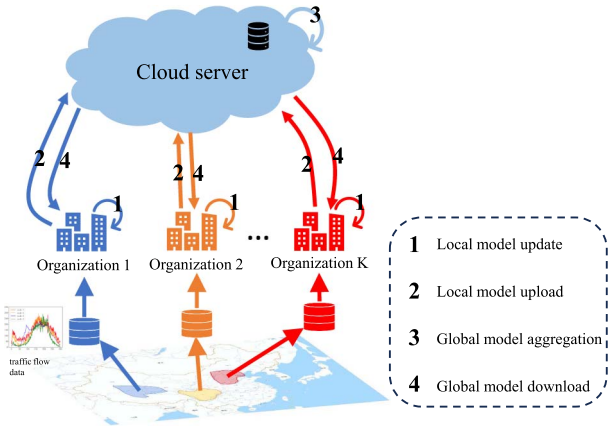

Fig. 2. Federated learning architecture diagram.

would be needed to train models with high accuracy, and the coordination between different organizations often involves data exchange, which also poses a risk of privacy leakage. To solve the above problems, we adopt federated learning, which is suitable for solving the data island problem due to its ability to train models without moving data. The process of federated learning is shown in Fig. 2. However, when introducing federated learning for traffic flow prediction, other problems come, which are heterogeneity challenges in both graph topology structures and data quantities. The challenges are brought by different organizations' deployment of sensors in different regions, affecting the size and values of V and E of the topology map as well as the quantities of data. The heterogeneity will slow down the convergence speeds, and reduce the accuracies and generalization abilities of the models.

C. Graph Transformer for Traffic Flow Prediction

The proposed traffic flow prediction model based on the Graph Transformer at the edge is shown in Fig. 3. The model mainly consists of an encoder-decoder module and a Graph Transformer Attention Network module that replaces the traditional adjacency matrix with an attention matrix. The reason for not using the traditional adjacency matrix is that it is a predefined, fixed structure that not only struggles to adapt to heterogeneous graphs but also fails to capture dynamic spatiotemporal relationships that change over time.

The reason for using Graph Transformer is that it has the following two benefits: 1) No requirement for a predefined adjacency matrix. Traditional graph convolution requires a predefined adjacency matrix for describing topological structure information. However, this method is limited when topological structure information cannot be obtained and is not conducive to training meta-learning models for different topological structures. 2) Ability to extract complete spatial-temporal information. Traditional graph convolution defines a fixed adjacency matrix during training, which cannot extract complex spatial-temporal information on changing spatial relationships over time. The following will introduce each component of the module.

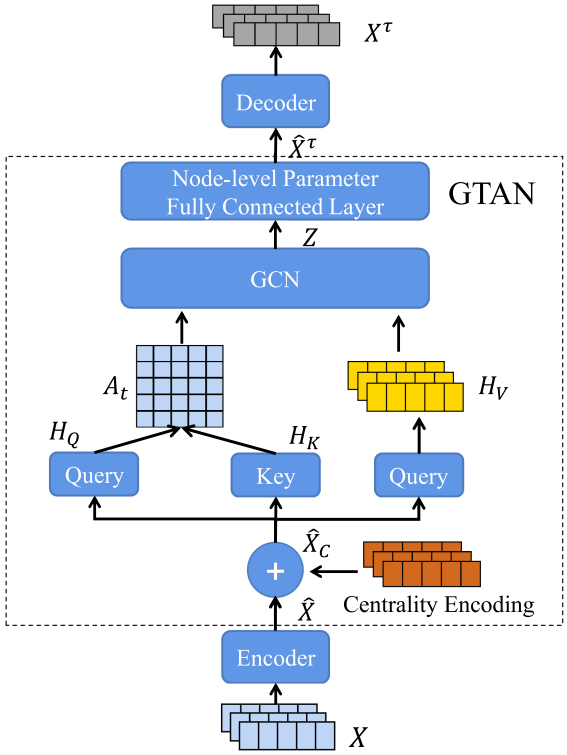

Fig. 3. The proposed local model. $A_t \in \mathbb{R}^{N\times N}$ is the node interaction matrix, and H_V denotes the output of value layer.

1) Graph Transformer Attention Network Module: We design GTAN to extract the spatio-temporal information of traffic flow sequences simultaneously on the basis of our former work [34].

Since the Transformer calculates attention distribution according to the nodes' relevance, the nodes' features are a very critical part when measuring the importance of nodes. Similarly to [35], we add the centrality encoding to the node features as the network input so that the attention mechanism can obtain deeper interaction patterns between a couple of nodes based on the query layer and key layer. Therefore, after the input data, the centrality encoding module processes $\hat{X}$, the attention layer can consider the influence of the traffic flow value of other locations in the traffic flow sequence on the location when processing the information of the current time position, and extract the spatio-temporal correlation of the entire traffic flow sequence, the formula is as follows:

$$H_Q = \hat{X}_C W_Q, H_K = \hat{X}_C W_K, H_V = \hat{X}_C W_V, \quad (1)$$

$$A_t = softmax\left(\frac{H_Q H_K^T}{\sqrt{d_K}}\right), \quad (2)$$

where $W_Q \in \mathbb{R}^{d\times d_k}$, $W_K \in \mathbb{R}^{d\times d_k}$ and $W_V \in \mathbb{R}^{d\times d_k}$ are the three learnable parameters in the attention layer, A_t is a matrix that captures the similarity between the query and the key, d_k denotes the hidden dimension of attention layer in the Transformer.

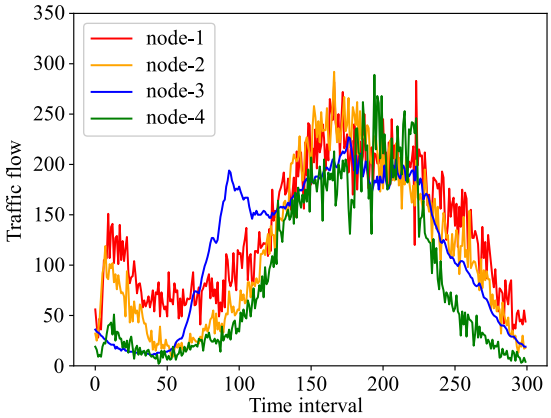

Fig. 4. Traffic flow of different nodes in the same period.

Unlike the traditional Transformer, our feed-forward layer is implemented based on GCN, and we take the attention matrix as the symmetric normalized form of the adjacency matrix to participate in the graph convolution operation. Unlike traditional adjacency matrices, which only consider the physical connections between nodes, attention matrices dynamically adjust the connections by calculating attention scores between each pair of nodes. This approach can better capture underlying structures and complex relationships in the data. Additionally, attention matrices are data-driven and can adaptively learn and adjust during training, enhancing the model's ability to represent and interpret data, so we can adaptively obtain hidden relationships in different traffic sequences. However, in traffic flow prediction, in addition to the spatial correlation caused by road distribution, the dynamic of time series and some other special factors (e.g., weather) make the spatial characteristics more complex. As shown in Fig. 4, different nodes may have different traffic trends in the same period. So, the main difference between the above implementation and traditional GCN is that we assign independent learnable parameters to each node rather than using shared parameters across all nodes. In the training process, each parameter can better fit each node's traffic flow change trend according to each node's spatio-temporal characteristics and obtain better prediction performance. Therefore, the operation can be expressed as follows:

$$Z = A_t H_V W + B, \tag{3}$$

where $W \in \mathbb{R}^{N\times C\times F}$ and $B \in \mathbb{R}^{N\times F}$ are learnable node-level parameters assigned for each node to capture the potential spatial features of nodes, N is the number of nodes in the traffic flow.

Finally, we apply a fully connected layer with node-level parameters to directly convert the output from $\mathbb{R}^{N\times F}$ to $\mathbb{R}^{N\times\tau}$ as the next τ steps prediction results of all nodes, that is:

$$\hat{X}^{\tau} = ZW_F + B_F, \tag{4}$$

where $\hat{X}^{\tau}$ is the prediction of the next τ steps, $W_F \in \mathbb{R}^{N\times F\times\tau}$ and $B_F \in \mathbb{R}^{N\times\tau}$ is the learnable node-level parameters of the final fully connected layer.

Compared with the previous method of separating temporal information and spatial information, the GTAN module considers the traffic information of all time slices of the entire traffic flow sequence and their spatial relationships, which makes the GTAN module better capture the deep information of the traffic flow sequence, so as to obtain more accurate prediction performance.

2) Encoder-Decoder Module: Since client data may have different graph structures and different numbers of nodes, input data sizes may vary. To address this, we use an encoder-decoder to standardize the data size.

The encoder performs dimension transformation on the input data before it enters GTAN. Specifically, it consists of a fully connected layer:

$$\hat{X} = w_E X + b_E. \tag{5}$$

The decoder maps the output of GTAN back to the input dimension and also consists of a fully connected layer as:

$$X^{\tau} = w_D \hat{X}^{\tau} + b_D, \tag{6}$$

where X represents the local data from the client, while X^{τ} represents the predicted values of the model. w_E, b_E, w_D and b_D are all trainable parameters. After sufficient training, the encoder can extract important features from the data during mapping that contribute to model prediction, while the decoder enhances the expressiveness of the model.

D. Graph Federated Meta-Learning FedGM for Traffic Flow Prediction

Inspired by the work of Fallah et al. [31], we combine federated learning and meta-learning to treat data with different graphics topologies for each client as a different training task. The problem of client-side graph topological heterogeneity will be an advantage for training meta-learning models with generalization performance. In addition, federated meta-learning can naturally complete the subsequent model personalization process by training a predictive model for each client that is suitable for local data. Traditional meta-learning models, such as the MAML model, perform several gradient updates on each task in the inner loop and then adjust the initial parameters through the outer loop. The model needs to switch between inner and outer loops and calculate second-order gradients, which results in a higher complexity for MAML. In contrast, we adopt the Reptile meta-learning model. Compared to traditional meta-learning algorithms, Reptile trains on multiple tasks, performing only one or a few gradient updates for each task, and then adjusts the model parameters based on the updates from all tasks. Moreover, Reptile adjusts the model parameters by averaging the gradient updates from different tasks, so it does not require second-order gradients, leading to a lower complexity. The advantage of fast training speed and fewer required parameters makes Reptile more suitable for combining with federated learning to construct FedGM.

The original algorithm of the Reptile is shown in algorithm 1, which includes the inner loop of steps 2-6 and the outer loop of step 7. By comparing it with the federated learning process, it can be observed that by treating the training data of each client as a task in Reptile, the training process from steps 2 to 6 can be implemented on the client side to do the inner loop. Step 7 can

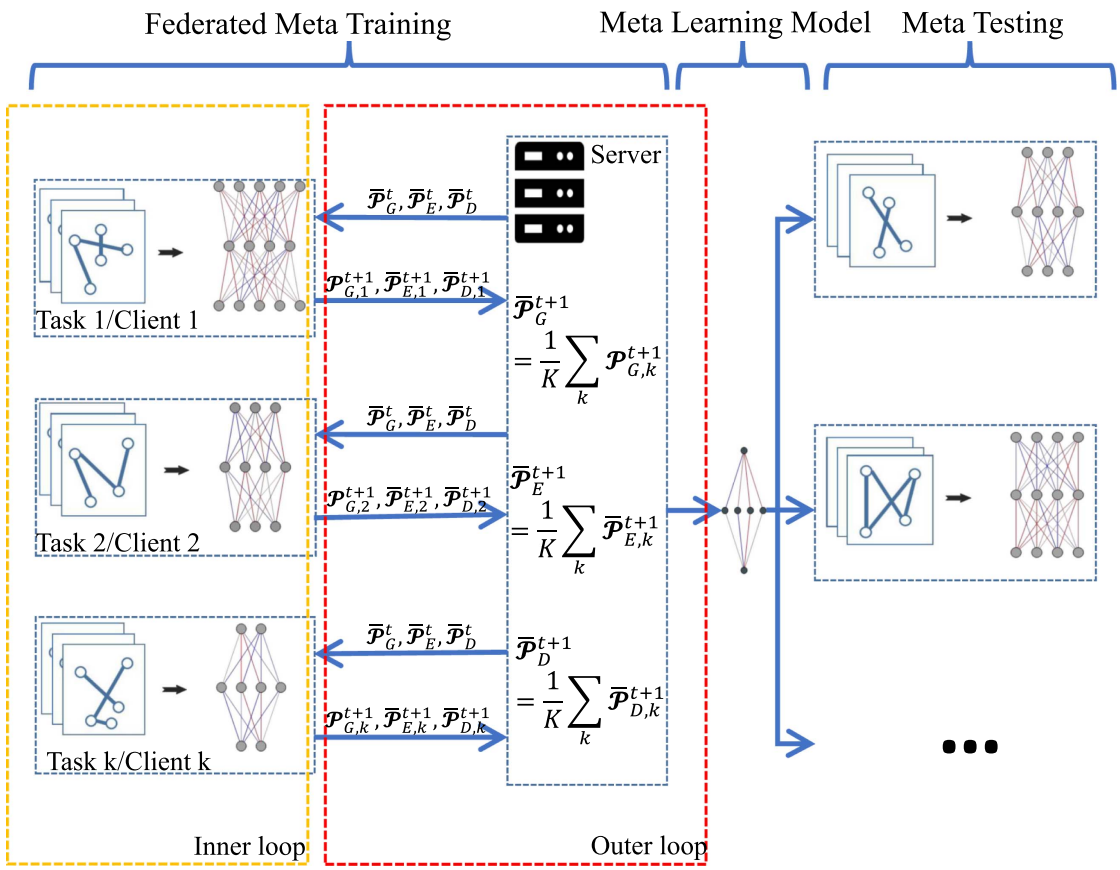

Fig. 5. The federated meta-learning process.

be implemented on the server to compute the average and to do the outer loop. This way, the Reptile algorithm can be realized through federated learning.

Corresponding to the training cycle of meta-learning, the proposed algorithm training process includes meta-training phases and meta-testing phases. As shown in Fig. 5, a server and multiple clients complete the meta-training process of graph-based federated meta-learning based on Reptile. Different graph structure data with different topologies from different clients are treated as separate training tasks, and each selected client first performs an inner loop locally and then uploads the model parameters to the cloud for an outer loop to determine the update direction of the optimal initialization parameters for a model that can quickly adapt to different tasks. After the optimal initialization parameters are trained, the initialized model, the meta-learning model, is sent to each client and then trained with local data to obtain a predictive model.

Due to the different number of nodes, different client initialization encoders and decoders are inconsistent in size and cannot participate in model aggregation. If the trained encoders and decoders are kept locally for the next round of training, not only can it resolve the aforementioned issue, but it can also preserve the parameters learned by the encoders and decoders, accelerating the convergence of the meta-training process. However, this approach has the following drawbacks. First, the participating client must be required to participate in each round of federated learning during the meta-training to update the state of the local private encoder and decoder. However, the general federated learning process extracts part of the client to participate in training every round to save training costs, at the same time, during the meta-training phase, only part of tasks is selected for the inner loop, this can lead to clients who are not selected in a given round being unable to update their local private encoding and decoding states. And second, this approach can render the trained global model unable to adapt to clients with an unknown graph topology, because they do not have a trained encoder

Algorithm 1: Reptile (Parallel Implementation).

Input initialize the model w^0, multiple task data x_T
Output meta-learning model w_T

```
randomly initialize the model w = w^0
while training rounds not completed do
    sample K tasks from x_T
    for each task
    while training rounds not completed do
        calculate gradients using the task dataset to update
        the model and obtain w_k
    end while
    w ← w + (1/K) Σ_{k=1}^{K} (w_k − w)
end while
obtain the meta-learning model w_T = w
```

and decoder. If the trained encoders and decoders are discarded during model aggregation, it will result in each client randomly initializing its encoder and decoder in every round, significantly reducing the model's convergence speed.

Based on the above problems, considering that the parameter means contains the training results of the encoder and decoder to some extent and can match the other model parameters after average aggregation, we propose a solution of using the parameter means of the encoder and decoder for aggregation. The optimal encoder and decoder parameter mean values are learned in the meta-training phase, and the means will be used to initialize the encoder and decoder in the meta-testing phase.

During the meta-training phase, each client receives the aggregated model parameters $\bar{\mathcal{P}}_G^t$ distributed by the server to initialize the local model GTAN at the current round. The encoder and decoder parameters are initialized to the mean values of the encoder and decoder distributed by the server, denoted by $\bar{\mathcal{P}}_E^t$ and $\bar{\mathcal{P}}_D^t$, respectively. The client then trains the local model using its own data. Each client then calculates the average of its local encoder and decoder parameters and prepares to upload them to the server for aggregation. (7) represents the process where the client uses local data to train the initial parameters distributed by the server to update its local models, while (8) illustrates the process where clients compute the average of local encoder-decoder parameters. (7) and (8) are both executed during the inner loop stage of meta-training.

$$\mathcal{P}_{G,k}^{t+1}, \mathcal{P}_{E,k}^{t+1}, \mathcal{P}_{D,k}^{t+1} \leftarrow ModelUpdate(D_k^t, \bar{\mathcal{P}}_G^t, \bar{\mathcal{P}}_E^t, \bar{\mathcal{P}}_D^t), \quad (7)$$

$$\bar{\mathcal{P}}_{E,k}^{t+1} = \frac{1}{M_{k,E}} \sum \mathcal{P}_{E,k}^{t+1}, \quad \bar{\mathcal{P}}_{D,k}^{t+1} = \frac{1}{M_{k,D}} \sum \mathcal{P}_{D,k}^{t+1}, \quad (8)$$

where $\mathcal{P}_E = \{w_E, b_E\}$ represents encoder parameters in (5), $\mathcal{P}_D = \{w_D, b_D\}$ represents decoder parameters in (6), and $\mathcal{P}_G = \{W_Q, W_K, W_V, W, B, W_F, B_F\}$ represents GTAN model parameters as shown in (1), (3) and (4). $ModelUpdate$ represents the process of updating GTAN, encoder, and decoder parameters with local data. The D_k^t represents local data from the kth client. $M_{k,E}, M_{k,D}$ are the number of encoder nodes and decoder nodes of the client k, respectively. $\mathcal{P}^t$ represents the parameters that were delivered to the client to participate in the current round of training after the cloud aggregation in the previous round t, and $\mathcal{P}^{t+1}$ represents the parameters that have been updated in the current round $t+1$ of local training.

When the local training is completed, the GTAN parameters $\mathcal{P}_{G,k}^{t+1}$, the mean values of encoder parameters $\bar{\mathcal{P}}_{E,k}^{t+1}$ and decoder parameters $\bar{\mathcal{P}}_{D,k}^{t+1}$ are uploaded to the server, then the server performs average aggregation, Equations (9) and (10) represent the process where the server aggregates the received parameters mentioned above. (9) and (10) are both executed during the outer loop stage of meta-training.

$$\bar{\mathcal{P}}_E^{t+1} = \frac{1}{K} \sum_k \bar{\mathcal{P}}_{E,k}^{t+1}, \quad \bar{\mathcal{P}}_D^{t+1} = \frac{1}{K} \sum_k \bar{\mathcal{P}}_{D,k}^{t+1}, \quad (9)$$

$$\bar{\mathcal{P}}_G^{t+1} = \frac{1}{K} \sum_k \mathcal{P}_{G,k}^{t+1}, \quad (10)$$

Algorithm 2: FedGM Meta-Training Process.

Input local data D_k
Output GTAN's optimal initial parameter $\bar{\mathcal{P}}_G$, best mean values of encoder and decoder $\bar{\mathcal{P}}_E$, $\bar{\mathcal{P}}_D$

while training rounds not completed **do**
  Client-side
  if The first round of training **then**
    randomly initialize $\bar{\mathcal{P}}_{G,k}^t$, $\bar{\mathcal{P}}_{E,k}^t$, $\bar{\mathcal{P}}_{D,k}^t$
  else
    receiving $\bar{\mathcal{P}}_G^t$, $\bar{\mathcal{P}}_E^t$, and $\bar{\mathcal{P}}_D^t$ send by the server
  end if
  while training rounds not completed **do**
    obtain $\mathcal{P}_{G,k}^{t+1}, \mathcal{P}_{E,k}^{t+1}, \mathcal{P}_{D,k}^{t+1}$ with D_k^t by (7) and (8)
  end while
  send $\mathcal{P}_{G,k}^{t+1}$, $\bar{\mathcal{P}}_{E,k}^{t+1}$, and $\bar{\mathcal{P}}_{D,k}^{t+1}$ to server
  Server-side
  receiving $\mathcal{P}_{G,k}^{t+1}$, $\bar{\mathcal{P}}_{E,k}^{t+1}$, and $\bar{\mathcal{P}}_{D,k}^{t+1}$ send by the client
  calculate $\bar{\mathcal{P}}_G^{t+1}$, $\bar{\mathcal{P}}_E^{t+1}$, $\bar{\mathcal{P}}_D^{t+1}$ by (9) and (10)
  send $\bar{\mathcal{P}}_G^{t+1}$, $\bar{\mathcal{P}}_E^{t+1}$, $\bar{\mathcal{P}}_D^{t+1}$ to the client
end while
output $\bar{\mathcal{P}}_G$, $\bar{\mathcal{P}}_E$ and $\bar{\mathcal{P}}_D$

where K is the number of clients participating in federated learning. Through the final aggregation, the server obtains the best GTAN parameters and the average values of encoders and decoders. The detailed training process of FedGM is shown in Algorithm 2. In the FedGM training process, meta-learning is conducted within a federated learning framework to ultimately obtain a meta-model.

The meta-training stage obtains the best initialization parameters. Then, the meta-learning model is further trained to get a model that can make predictions after the meta-testing stage. Clients participating in the meta-testing phase can use data with known topological structures used in federated learning for training or data with unknown topological structures not used in federated learning. First, the local model is initialized using the parameters of the GTAN, the mean constants of the encoder and decoder learned by meta-learning, respectively, and then the model is trained until convergence using the local data, and the final model can be used for prediction tasks.

IV. Experiment and Analysis

A. Experimental Design

Dataset: We adopt the real-world traffic flow data set PeMSD4, collected by the Performance Measurement System (PeMS) built by the California Department of Transportation. The PeMSD4 dataset includes 307 sensor nodes and collects traffic flow data of three types: traffic volume, road occupancy, and average vehicle speed, in the San Francisco Bay Area from January 1, 2018, to February 28, 2018. The real-time data are collected every 30 seconds and aggregated every 5 minutes.

Data pre-processing: We use the data from the previous hour as input to predict the traffic volume data for the next hour. Since data is collected every five minutes, 12-time points traffic volume

TABLE I
THE NUMBER OF SENSOR NODES HELD BY CLIENTS PARTICIPATING IN META-LEARNING

	Client 1	Client 2	Client 3	Client 4	Client 5	Client 6	Client 7
Nodes	25	36	30	45	20	33	22

TABLE II
THE NUMBER OF SENSOR NODES HELD BY CLIENTS THAT ARE NOT PARTICIPATING IN META-LEARNING BUT PARTICIPATING IN META-TESTING

	Client 8	Client 9	Client 10
Nodes	20	30	40

data are used as the model input and output. For the missing data in the dataset caused by various reasons, linear interpolation is used to fill in the gaps.

Accuracy metrics: To validate the performance of the proposed model, we use Mean Absolute Error (MAE) and Root Mean Square Error (RMSE) as evaluation metrics for the experiment.

Mean Absolute Error (MAE): It is the mean of the absolute differences between the predicted and actual values, calculated by taking the absolute difference of predicted and actual values and averaging over the samples, as

$$\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n}|\hat{y}_i - y_i|, \tag{11}$$

where $\hat{y}$ represents the predicted value, n represents the number of samples, and y represents the actual value. The smaller the MAE value, the closer the predicted traffic volume is to the actual value.

Root Mean Square Error (RMSE): It is calculated by taking the square root of the mean of the squared differences between the predicted and actual values as

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2}. \tag{12}$$

The smaller the RMSE value, the more accurate the predicted results are.

Client configuration: In the experiment, the first 301 nodes out of 307in the dataset were partitioned into 10 groups based on their locations, and each client in a certain region holds 1 group to simulate the distributed data distribution in federated learning. The regions do not overlap. The way of partition is to ensure that the sensor data held by a client are spatially correlated, which consists with the reality. As shown in Table I, the number of nodes owned by each client varies, with clients 1 to 7 participating in federated learning. As shown in Table II, clients 8 to 10 did not participate in federated learning and were used to validate the generalization performance of the meta-learning model.

Experimental parameters: This section implements the proposed algorithm and comparison algorithms by PyTorch framework version 1.8.0. Each client holds 2500 data samples, and the test set contains 2500 samples. There are 50 communication rounds, 10 local training rounds, a batch size of 256, and a learning rate of 0.001. For FedRecon, 500 samples are partitioned into the support set, and 2000 samples are partitioned into the query set. During local training, GTAN is frozen using the support set and the encoder and decoder are trained for 200 rounds. Then, the entire model is trained for 10 rounds using the validation set. The optimizer used is Adam, and the loss function used is mean square error (MSE), as

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2. \tag{13}$$

B. Performance of GTAN

We compare the performance of GTAN model with the following latest traffic prediction models, which are

- Historical Average (HA): HA takes the average value of the last 12 time slices as the prediction value of the following time slices.
- Vector Auto-Regression (VAR) [6]: VAR can capture the spatial features among all the series to predict the following-time value.
- DSANet [36]: DSANet is a time series prediction model using CNN to capture the temporal correlations and self-attention mechanism to capture the spatial correlations.
- DCRNN [1]: DCRNN combines GCN with recurrent models in an encoder-decoder manner to tackle forecasting problems.
- STGCN [2]: STGCN is a spatio-temporal graph convolutional network that applies GCN in spatial feature extraction and CNN in capturing temporal features.
- ASTGCN [21]: ASTGCN adds an attention mechanism to the spatial-temporal graph convolutional network and gets better performance in prediction tasks.
- STSGCN [22]: STSGCN stacks multiple localized GCN layers with adjacent matrices over the time axis to capture the spatial-temporal correlations.
- STTN [24]: STTN leverages dynamically directed spatial dependencies and long-range temporal dependencies by Transformer to improve the accuracy of long-term traffic forecasting.

1) Performance Comparisons With Different Modules: To evaluate the proposed Graph-Transformer's performance and the node-level parameter forward propagation module, we make a comprehensive ablation study. We construct GCN-N by removing the Transformer structure in our model and using the predefined adjacent matrix as a graph convolution matrix in GCN. GTN is a variant of our GTAN. In comparison to GTAN, GTN does not include fully connected layers in the feed-forward network. Parameters are shared among each node instead of being individually assigned to each node. The experiment on the PeMSD4 dataset is illustrated in Table III. It can be observed that the prediction performance of GTN and GTAN models with Graph-Transformer modules is significantly better than that of traditional graph convolutional networks using predefined adjacency matrices. This shows that the proposed GraphTransformer module can capture the spatial correlation of the traffic flow sequence and the temporal correlation synchronously. The results also indicate that it can effectively improve the prediction

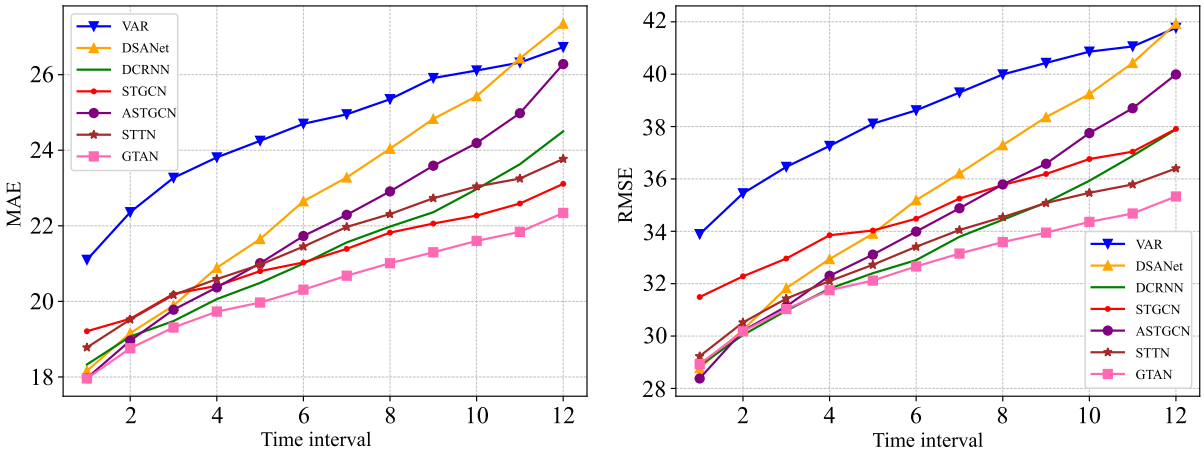

Fig. 6. Prediction performance comparison at each time step on PeMSD4.

TABLE III
ABLATION STUDY RESULTS OF GTAN ON PEMSD4

Model	5min		10min		30min		60min	
	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE
GCN-N	41.14	64.93	41.40	65.13	42.54	66.12	44.49	68.03
GTN	18.24	29.12	18.63	30.37	19.63	31.87	20.98	32.95
GTAN	**17.95**	**28.90**	**18.34**	**29.54**	**19.47**	**31.12**	**20.38**	**32.66**

TABLE IV
THE ACCURACY OF THE META-TRAINING MODEL IN THE DATA ON EACH CLIENT

Model	PeMSD4	
	MAE	RMSE
HA	37.87	59.13
VAR	24.32	37.95
DSANet	22.84	35.87
DCRNN	23.76	35.77
STGCN	21.86	35.12
ASTGCN	22.93	35.22
STTN	21.55	33.48
STSGCN	21.19	33.65
GTAN	20.38	32.66

performance of MAE by assigning individual parameters for each node.

2) Performance Comparisons With the State-of-The-Art Methods: Table IV shows the average prediction performance over the next hour. It can be seen from Table IV that the traditional approaches (i.e., HA and VAR) usually have an unsatisfactory prediction result due to the limitation of the ability to deal with traffic data's nonlinearity and complex spatio-temporal characteristics. In contrast, deep learning-based methods perform better than traditional time series analysis methods. Several state-of-the-art methods that consider the temporal and spatial correlation are more effective than the other methods, such as STGCN, ASTGCN, and STSGCN. The transformer-based methods like STTN also achieve a good prediction performance. Unlike the above method, our GTAN model achieves better traffic flow prediction performance than all the above methods because the Graph-Transformer module can comprehensively consider the information of all moments of the traffic flow sequence and synchronously extract their deep spatio-temporal features. In addition to predicting the average performance of traffic flow over the next hour, we also compare the average MAE and RMSE of each model when predicting traffic flow from the next 5 minutes to the next 60 minutes on the PeMSD4 dataset, as shown in Fig. 6. It can be seen from the figure that the prediction performance of each model for short-term traffic flow prediction tasks is similar, but with the increase in prediction time, the GTAN model has significantly better prediction performance than other models.

TABLE V
THE COMPUTATION COST ON THE PEMSD4 DATASET

Model	Parameters	Training Time/epoch (s)
DCRNN	149057	74.38
STGCN	211596	35.21
ASTGCN	450031	108.14
STTN	511261	155.39
GTAN	3332009	19.96

3) Computation Cost Comparisons With Different Methods: We also evaluate the computation cost of our model compared with DCRNN, STGCN, ASTGCN, and STTN on the PeMSD4 dataset. As shown in Table V, due to the temporal convolution structure of STGCN, it has a shorter running time and fewer parameters. ASTGCN adds an attention mechanism to the model, adding more trainable parameters and complex computations, resulting in much longer training times for ASTGCN. STTN is the slowest to train because it builds two complex graph convolution and Transformer modules to independently process the spatial and temporal features of traffic flow data and finally requires additional computation to fuse the features to make predictions. The GTAN model takes the shortest time to train. Although the amount of parameters of the GTAN model is 6-20 times that of other models, the model only contains a layer of Transformer structure that is more concise and involves fewer operations than other algorithms, so GTAN training speed is faster than other comparison algorithms. In general, although GTAN requires a large memory to store training parameters, it has significant advantages over other methods in terms of training speed.

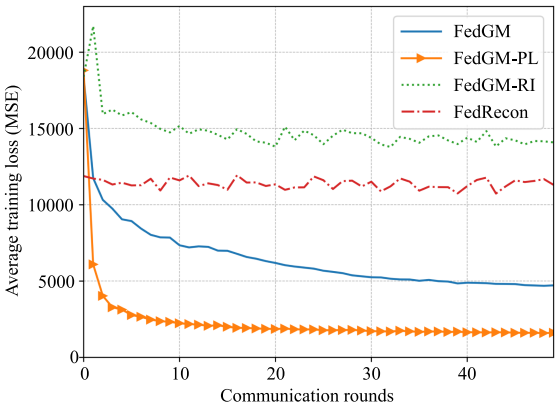

Fig. 7. The loss of meta-training via communication rounds.

TABLE VI
THE ACCURACY OF THE META-TRAINING MODEL IN THE DATA ON EACH CLIENT

	FedGM	FedRecon	FedGM-RI	FedGM-PL	RI
Client 1	46.59	67.96	71.54	**36.57**	134.37
Client 2	46.91	83.74	99.16	**41.02**	163.32
Client 3	43.07	84.01	94.95	**29.22**	157.06
Client 4	48.01	73.53	105.39	**41.07**	187.05
Client 5	39.49	65.44	77.33	**35.07**	135.38
Client 6	35.55	53.52	68.58	**23.77**	135.64
Client 7	46.04	60.17	76.97	**22.29**	131.14
Client 8	**47.57**	54.07	70.19	98.57	115.95
Client 9	**42.01**	52.89	75.59	106.98	133.03
Client 10	**49.98**	85.97	77.75	119.67	140.69
Mean	**44.52**	68.13	81.75	55.42	143.36

C. Performance of the FedGM

To assess the effectiveness of the proposed FedGM, the following algorithms are used for comparison.

- FedGM-RI: The encoder and decoder are randomly initialized at each training iteration.
- FedGM-PL: Inspired by the work of Collins et al. [37], the parameters of the encoder and decoder are kept as private on clients and are not aggregated.
- FedRecon: A federated learning algorithm proposed by Singhal et al. [38], which divides local training data into support set and query set, separates the model into private and public layers, and modifies the local training process to pre-train the private layer using the support set, followed by training the entire model using the query set. FedRecon solves the problem of preserving private layer parameters. In this experiment, the encoder and decoder are private layers, and GTAN is the public layer in FedRecon.
- LM: A local model trained only with local data.

1) Meta-Training. The loss of meta-training: In the meta-training process, we propose an encoder-decoder mean aggregation method to accelerate convergence under the ensurance of the effectiveness of the proposed algorithm. As shown in Fig. 7, the average training loss of the clients during the meta-training process decreases with communication rounds, where the vertical axis represents the average MSE of the globally aggregated model on each client. It can be observed in Fig. 7 that FedGM has consistently lower losses than FedGM-RI (which can be treated as an ablation algorithm of FedGM without using encoder-decoder mean aggregation) during training. For example, in the 10th round, the loss of FedGM was only 53.23% of FedGM-RI. By the 30th round, the loss of FedGM-RI had stabilized and no longer decreased, while the loss of FedGM still showed a significant downward trend. This indicates that FedGM-RI is more susceptible to the negative effects of data heterogeneity, making it difficult for its aggregated model to update to the global optimum. In contrast, FedGM demonstrates faster convergence speed and lower convergence loss through encoder-decoder mean aggregation. Additionally, FedRecon calculates the loss based on the pre-trained encoder-decoder results, allowing it to achieve relatively low loss in the early stages of training. However, the lack of a decreasing trend in its loss suggests that it is unable to learn the common knowledge among different topological structures of data in different communication rounds. Notably, FedGM-PL achieved the fastest convergence speed and the lowest convergence loss because the parameters learned by the encoder-decoder can be fully preserved across communication rounds. However, this method lacks generalization for unknown graph topology data, which will be verified in later experiments.

The MAE of the meta-training model on each client: FedGM aims to train the best initial parameters that can adapt to different graph topologies. To this end, this experiment tests the accuracy of the meta-training model on each client, including clients 1 to 7 who participates in federated learning and clients 8 to 10 with unknown graph topologies. Table VI shows the results, with bold indicating the lowest MAE.

FedGM-PL uses the encoder-decoder that has been fully trained through federated learning on clients 1 to 7, achieving the lowest MAE on them. However, for clients 8 to 10 that did not participate in federated learning and thus had randomly initialized encoders, the encoder and decoder cannot match the other trained model parameters, resulting in poor performance. This experiment shows that retaining encoder-decoder parameters during meta-training does not lead to a generalized model. For FedGM-RI, the encoder-decoder is randomly initialized in each round of training, allowing the other parts of the model to adapt to the encoder-decoder's randomly initialized parameters, giving it some adaptability for non-federated clients, while the average MAE on federated clients is 52.31 higher than that of FedGM-PL. RI is an untrained model that has the lowest accuracy on all clients. FedRecon tests the performance of the encoder-decoder after pre-training and thus achieves a relatively low MAE. Although FedGM has an average MAE 10.94 higher than that of FedGM-PL on federated clients, it achieves the best accuracy on non-federated clients and has the lowest average MAE on all clients. The experiment demonstrates that FedGM can train the best initial model for different topological structure data.

2) Meta-Testing: The meta-testing stage involves the personalized model training based on the meta-training model. To ensure a fair comparison, FedGM-PL does not use the fully trained encoder-decoder in subsequent experiments. Although

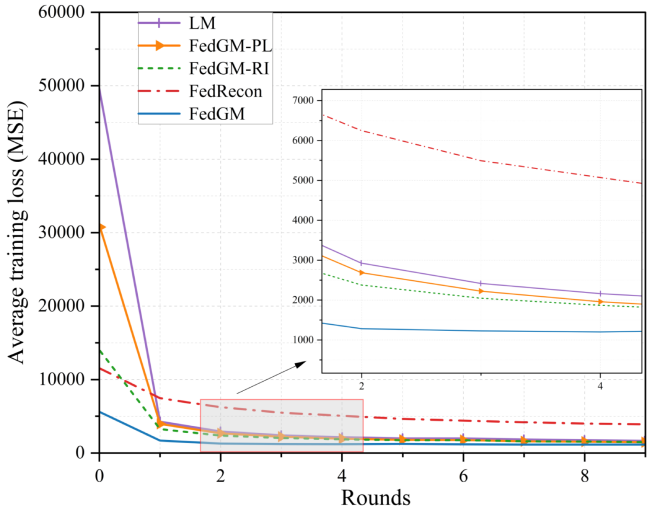

Fig. 8. The loss during meta-testing with training rounds.

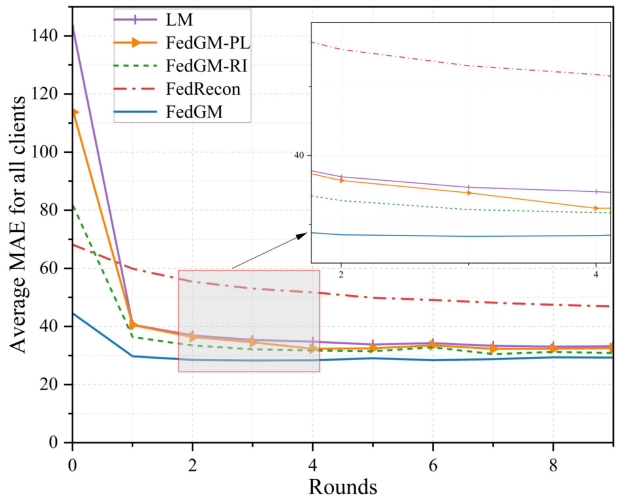

Fig. 9. The meta-test process of MAE varies with the training round.

FedGM-PL has low generalization ability, it may over-fit the local data of the clients 1 to 7 with the fully trained encoder-decoder, leading to the lowest convergence time and highest prediction accuracy on them, which would be unfair for FedGM.

The loss of meta-testing with training rounds: This experiment is performed on ten clients, where the loss is the average of the training loss of the ten clients. Experimental results are shown in Fig. 8. Fig. 8 shows that the initial loss on a randomly initialized model (e.g., LM) is relatively high. However, after being trained through federated meta-learning, FedGM, FedGM-PL, and FedGM-RI achieve lower initial loss and faster convergence speed than the randomly initialized model. The final convergent loss is also lower than that of the randomized initialized model, demonstrating that federated meta-learning can learn general features and knowledge adaptable to different topological structures during training on diverse topological data, enabling a better initial position to start training a better personalized model. Additionally, Fig. 8 also demonstrates FedGM has the lowest initial loss, more than half lower than the second-ranked model. The zoomed-in plot clearly shows that By the second training round, FedGM has already reached convergence, with the lowest final loss among the five models. While FedRecon obtains lower initial loss by relying on the pre-training encoder-decoder, it fails to achieve a low MSE in subsequent training rounds.

The accuracy of meta-testing with training rounds: This experiment tests the changes in prediction accuracy of the personalized model as training rounds increase. From Fig. 9, it can be observed that FedGM achieved the highest accuracy after just two rounds of training, as shown in the zoomed-in plot, indicating that the meta-learning model can quickly adapt to data with different topological structures through the meta-training process. In addition, FedGM maintains the highest accuracy in the final round compared with other algorithms, which all have lower convergence rates and higher MAE. That validates the effectiveness and advancement of FedGM on learning the optimal initialization model.

The highest accuracy of each algorithm in each client: This experiment tests the highest achievable accuracy of each algorithm on different clients after meta-testing. The results are shown in Table VII, with boldface indicating the lowest MAE and RMSE.

Table VII shows that after being trained through federated meta-learning, FedGM, FedGM-RI, and FedGM-PL achieve higher accuracy than the locally trained model LM. FedGM-RI, as a factor analysis technique of FedGM, cannot retain the knowledge obtained through the encoder-decoder training process during federated meta-training, resulting in lower accuracy than FedGM in meta-testing. FedGM-PL can achieve relatively good accuracy on participating clients for retaining private layers. However, if discarding the trained private layers for the purpose of training the best initial parameters adaptable to unknown graph topological data (in other words, the encoder-decoder is randomly initialized in each round to enhance generalization), FedGM-PL performs worse than FedGM on each client. FedRecon obtained lower accuracy than the randomly initialized local model, indicating that the encoder-decoder, although pre-trained, cannot obtain a model that matches GTAN and can cause the model to get trapped in a local optimum. These results show that FedRecon is not suitable for learning common knowledge of different graph topological structures. FedGM achieved the best prediction results on each client except for client 6. Comparing with FedRecon, FedGM-RI, FedGM-PL, and LM, the average MAE of FedGM decreases by 19.73, 2.95, 3.56, and 5.29, respectively, while the average RMSE decreases by 25.04, 4.25, 5.27, and 7.43, respectively. These experiments fully demonstrate that the meta-learning model trained through FedGM can achieve the highest accuracy after being trained on local data.

V. Conclusion and Future Work

In this paper, we propose a GTAN model that combines the advantages of Transformer and GCN to capture the spatio-temporal correlation of traffic sequences and learn its unique feature representation at each node based on the data characteristics for traffic flow prediction. To address the issue of data silos in traffic flow data, we propose a graph federated meta-learning approach

TABLE VII
THE ACCURACY OF EACH ALGORITHM

	FedGM		FedRecon		FedGM-RI		FedGM-PL		LM	
	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE
Client 1	**29.77**	**46.21**	48.79	67.67	32.47	50.25	38.97	60.78	35.48	55.48
Client 2	**34.28**	**53.24**	54.27	76.4	34.63	54.83	36.08	55.52	35.22	53.45
Client 3	**28.93**	**41.29**	50.95	69.99	33.83	49.29	30.41	43.81	40.41	59.6
Client 4	**28.78**	**42.79**	50.72	70.08	32.95	49.64	36.42	52.7	43.58	62.72
Client 5	**30.57**	**47.63**	50.14	70	34.67	50.92	31.53	46.33	33.97	52.77
Client 6	23.55	33.81	39.22	52.79	25.1	35.67	**21.99**	**32.49**	25.55	37.48
Client 7	**22.21**	**37.73**	38.43	56.18	25.5	42.21	35.23	59.97	28.08	42.81
Client 8	**21.58**	**31.61**	39.49	57.68	25.09	37.03	22.16	31.86	23.79	34.94
Client 9	**22.06**	**31.23**	36.7	50.64	23.94	33.82	22.98	32.46	24.5	34.84
Client 10	**30.35**	**44.34**	60.72	88.83	33.46	48.69	31.93	46.65	34.42	50.14
Mean	**27.21**	**40.99**	46.94	66.03	30.16	45.24	30.77	46.26	32.5	48.42

called FedGM for fast learning of different graph topologies during both training and testing. We adapt and aggregate heterogeneous node quantities using encoder-decoders, with mean aggregation used to accelerate convergence. Additionally, we validate the effectiveness of our proposed approach using the PeMSD4 real-world traffic flow dataset. Experimental results demonstrate that compared with advanced traffic flow prediction methods, GTAN achieves higher prediction accuracy, while FedGM can train a meta-learning model that quickly adapts to different graph topologies no matter the client participates the model training or not.In addition, FedGM achieves higher prediction accuracy in a shorter time than competitive models after meta-testing.

Based on the existing work, on one hand, we will further consider the interaction of traffic flow data owned by different clients in topology and feature space, and use the physical connectivity and model similarity relationship between clients to update the model parameters; on the other hand, considering the limitation of communication and computational resources in federated learning, we try to keep the the personalized part of each client's model while pruning the other parts to obtain personalized sparse models.

REFERENCES

[1] Y. Li, R. Yu, C. Shahabi, and Y. Liu, "Diffusion convolutional recurrent neural network: Data-driven traffic forecasting," in *Proc. 6th Int. Conf. Learn. Representations*, ICLR, Vancouver, BC, Canada, Conference Track Proceedings, Apr./May 2018. [Online]. Available: https://openreview.net/forum?id=SJiHXGWAZ

[2] B. Yu, H. Yin, and Z. Zhu, "Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting," in *Proc. 27th Int. Joint Conf. Artif. Intell.*, 2018, pp. 3634–3640.

[3] P. Kairouz et al., "Advances and open problems in federated learning," *Found. Trends Mach. Learn.*, vol. 14, no. 1, pp. 1–210, 2021.

[4] J. Liu and W. Guan, "A summary of traffic flow forecasting methods," *J. Highway Transp. Res. Develop.*, vol. 21, no. 3, pp. 82–85, 2004.

[5] M. M. Hamed, H. R. Al-Masaeid, and Z. M. B. Said, "Short-term prediction of traffic volume in urban arterials," *J. Transp. Eng.*, vol. 121, no. 3, pp. 249–254, 1995.

[6] K. Holden, "Vector auto regression modeling and forecasting," *J. Forecasting*, vol. 14, no. 3, pp. 159–166, 1995.

[7] Y. Wu, H. Tan, L. Qin, B. Ran, and Z. Jiang, "A hybrid deep learning based traffic flow prediction method and its understanding," *Transp. Res. Part C: Emerg. Technol.*, vol. 90, pp. 166–180, 2018.

[8] H. Yao et al., "Deep multi-view spatial-temporal network for taxi demand prediction," in *Proc. AAAI Conf. Artif. Intell.*, 2018, vol. 32, no. 1, pp. 2588–2595.

[9] X. Tang, H. Yao, Y. Sun, C. Aggarwal, P. Mitra, and S. Wang, "Joint modeling of local and global temporal dynamics for multivariate time series forecasting with missing values," in *Proc. AAAI Conf. Artif. Intell.*, 2020, vol. 34, no. 4, pp. 5956–5963.

[10] F. Zhao, G.-Q. Zeng, and K.-D. Lu, "EnLSTM-WPEO: Short-term traffic flow prediction by ensemble LSTM, NNCT weight integration, and population extremal optimization," *IEEE Trans. Veh. Technol.*, vol. 69, no. 1, pp. 101–113, Jan. 2020.

[11] L. Lin, W. Li, H. Bi, and L. Qin, "Vehicle trajectory prediction using LSTMS with spatial–temporal attention mechanisms," *IEEE Intell. Transp. Syst. Mag.*, vol. 14, no. 2, pp. 197–208, Mar./Apr. 2022.

[12] C. Li, L. Bai, W. Liu, L. Yao, and S. T. Waller, "Knowledge adaption for demand prediction based on multi-task memory neural network," in *Proc. 29th ACM Int. Conf. Inf. Knowl. Manage.*, 2020, pp. 715–724.

[13] L. Chen, W. Shao, M. Lv, W. Chen, Y. Zhang, and C. Yang, "AARGNN: An attentive attributed recurrent graph neural network for traffic flow prediction considering multiple dynamic factors," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 10, pp. 17201–17211, Oct. 2022.

[14] Q. Shi and H. Zhang, "An improved learning-based LSTM approach for lane change intention prediction subject to imbalanced data," *Transp. Res. Part C: Emerg. Technol.*, vol. 133, 2021, Art. no. 103414.

[15] G. Lai, W.-C. Chang, Y. Yang, and H. Liu, "Modeling long- and short-term temporal patterns with deep neural networks," in *Proc. 41st Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval*, 2018, pp. 95–104.

[16] J. Zhang, H. Che, F. Chen, W. Ma, and Z. He, "Short-term origin-destination demand prediction in urban rail transit systems: A channel-wise attentive split-convolutional neural network method," *Transp. Res. Part C: Emerg. Technol.*, vol. 124, 2021, Art. no. 102928.

[17] S. Bai, J. Z. Kolter, and V. Koltun, "An empirical evaluation of generic convolutional and recurrent networks for sequence modeling," 2018, *arXiv:1803.01271*.

[18] Y. Zhang, S. Wang, B. Chen, J. Cao, and Z. Huang, "TrafficGAN: Network-scale deep traffic prediction with generative adversarial nets," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 1, pp. 219–230, Jan. 2021.

[19] R. Sen, H.-F. Yu, and I. S. Dhillon, "Think globally, act locally: A deep neural network approach to high-dimensional time series forecasting," in *Proc. 33 rd Int. Conf. Neural Inf. Process. Syst.*, 2019, pp. 4837–4846.

[20] K. Niu, C. Cheng, J. Chang, H. Zhang, and T. Zhou, "Real-time taxi-passenger prediction with L-CNN," *IEEE Trans. Veh. Technol.*, vol. 68, no. 5, pp. 4122–4129, May 2019.

[21] S. Guo, Y. Lin, N. Feng, C. Song, and H. Wan, "Attention based spatial-temporal graph convolutional networks for traffic flow forecasting," in *Proc. AAAI Conf. Artif. Intell.*, 2019, vol. 33, no. 1, pp. 922–929.

[22] C. Song, Y. Lin, S. Guo, and H. Wan, "Spatial-temporal synchronous graph convolutional networks: A new framework for spatial-temporal network data forecasting," in *Proc. AAAI Conf. Artif. Intell.*, 2020, vol. 34, no. 1, pp. 914–921.

[23] X. Wang et al., "Traffic flow prediction via spatial temporal graph neural network," in *Proc. 2020 Web Conf.*, 2020, pp. 1082–1092.

[24] M. Xu et al., "Spatial-temporal transformer networks for traffic flow forecasting," 2020, *arXiv:2001.02908*.

[25] Y. Duan, N. Chen, S. Shen, P. Zhang, Y. Qu, and S. Yu, "FDSA-STG: Fully dynamic self-attention spatio-temporal graph networks for intelligent traffic flow prediction," *IEEE Trans. Veh. Technol.*, vol. 71, no. 9, pp. 9250–9260, Sep. 2022.

[26] Y. Zhao, M. Li, L. Lai, N. Suda, D. Civin, and V. Chandra, "Federated learning with non-IID data," 2018, *arXiv:1806.00582*.
[27] Y. Fraboni, R. Vidal, L. Kameni, and M. Lorenzi, "Clustered sampling: Low-variance and improved representativity for clients selection in federated learning," in *Proc. 38th Int. Conf. Mach. Learn.*, 2021, pp. 3407–3416.
[28] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proc. Mach. Learn. Syst.*, 2020, pp. 429–450.
[29] H.-Y. Chen and W.-L. Chao, "Fedbe: Making Bayesian model ensemble applicable to federated learning," in *Proc. Int. Conf. Learn. Representations*, 2021. [Online]. Available: https://openreview.net/forum?id=dgtpE6gKjHn
[30] V. Smith, C.-K. Chiang, M. Sanjabi, and A. Talwalkar, "Federated multi-task learning," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 4427–4437.
[31] A. Fallah, A. Mokhtari, and A. Ozdaglar, "Personalized federated learning with theoretical guarantees: A model-agnostic meta-learning approach," in *Proc. 34th Int. Conf. Neural Inf. Process. Syst.*, 2020, pp. 3557–3568.
[32] L. Yang, J. Huang, W. Lin, and J. Cao, "Personalized federated learning on non-IID data via group-based meta-learning," *ACM Trans. Knowl. Discov. Data*, vol. 17, no. 4, pp. 1–20, 2023.
[33] A. Shamsian, A. Navon, E. Fetaya, and G. Chechik, "Personalized federated learning using hypernetworks," in *Proc. 38th Int. Conf. Mach. Learn.*, 2021, pp. 9489–9502.
[34] H. Ruan, X. Feng, and H. Zheng, "Graph transformer attention networks for traffic flow prediction," in *Proc. IEEE 7th Int. Conf. Comput. Commun.*, 2021, pp. 1778–1782.
[35] C. Ying et al., "Do transformers really perform badly for graph representation?," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, vol. 34, pp. 28877–28888.
[36] S. Huang, D. Wang, X. Wu, and A. Tang, "DSANet: Dual self-attention network for multivariate time series forecasting," in *Proc. 28th ACM Int. Conf. Inf. Knowl. Manage.*, 2019, pp. 2129–2132.
[37] L. Collins, H. Hassani, A. Mokhtari, and S. Shakkottai, "Exploiting shared representations for personalized federated learning," in *Proc. 38th Int. Conf. Mach. Learn.*, 2021, pp. 2089–2099.
[38] K. Singhal, H. Sidahmed, Z. Garrett, S. Wu, J. Rush, and S. Prakash, "Federated reconstruction: Partially local federated learning," *Adv. Neural Inf. Process. Syst.*, vol. 34, pp. 11220–11232, 2021.

Xinxin Feng (Member, IEEE) received the B. Eng. and M. S. degrees in communication engineering from the Nanjing University of Science and Technology, Nanjing, China, in 2006 and 2008, respectively, and the Ph.D. degree in information and communication engineering from Shanghai Jiao Tong University, Shanghai, China, in 2015. From 2018 to 2019, she was a Visiting Scholar with Arizona State University, Tempe, AZ, USA. She is currently an Associate Professor with the College of Physics and Information Engineering, Fuzhou University, Fuzhou, China. Her research interests include machine learning, Big Data analytics, and Internet of Vehicles.

Haoran Sun received the B.Eng. degree in electronic information engineering from the Qingdao University of Technology, Qingdao, China, in 2022. He is currently working toward the M.S. degree in new-generation electronic information technology with Fuzhou University, Fuzhou, China. His research interests include multi-modal learning, federated learning and Internet of Vehicles.

Shunjian Liu received the B.S. degree in communication engineering from the Dongguan University of Technology, Dongguan, China, in 2020, and the M.S. degree in new-generation electronic information technology from Fuzhou University, Fuzhou, China, in 2023. His research interests include personalized federated learning, meta-learning, and Big Data analysis in Internet of Vehicles.

Junxin Guo received the B.S. degree in communication engineering from Fujian Normal University, Fuzhou, China, in 2021, and the M.S. degree in new-generation electronic information technology from Fuzhou University, Fuzhou, in 2024. His research interests include federated learning, deep learning, and Internet of Vehicles.

Haifeng Zheng (Senior Member, IEEE) received the B.Eng. and M.S. degrees in communication engineering from Fuzhou University, Fuzhou, China, in 2000 and 2003, respectively, and the Ph.D. degree in communication and information system from Shanghai Jiao Tong University, Shanghai, China, in 2014. From 2015 to 2016, he was a Visiting Scholar with the State University of New York, Buffalo, NY, USA. He is currently a Professor with the College of Physics and Information Engineering, Fuzhou University. His research interests include tensor theory, machine learning, Internet of Things, and wireless networks.